Call Centre Business Intelligence System (CCBIS)

Part 1 – Data Warehousing and Call Centre Performance Dashboard

Presented by Gang Huo

March 11, 2026

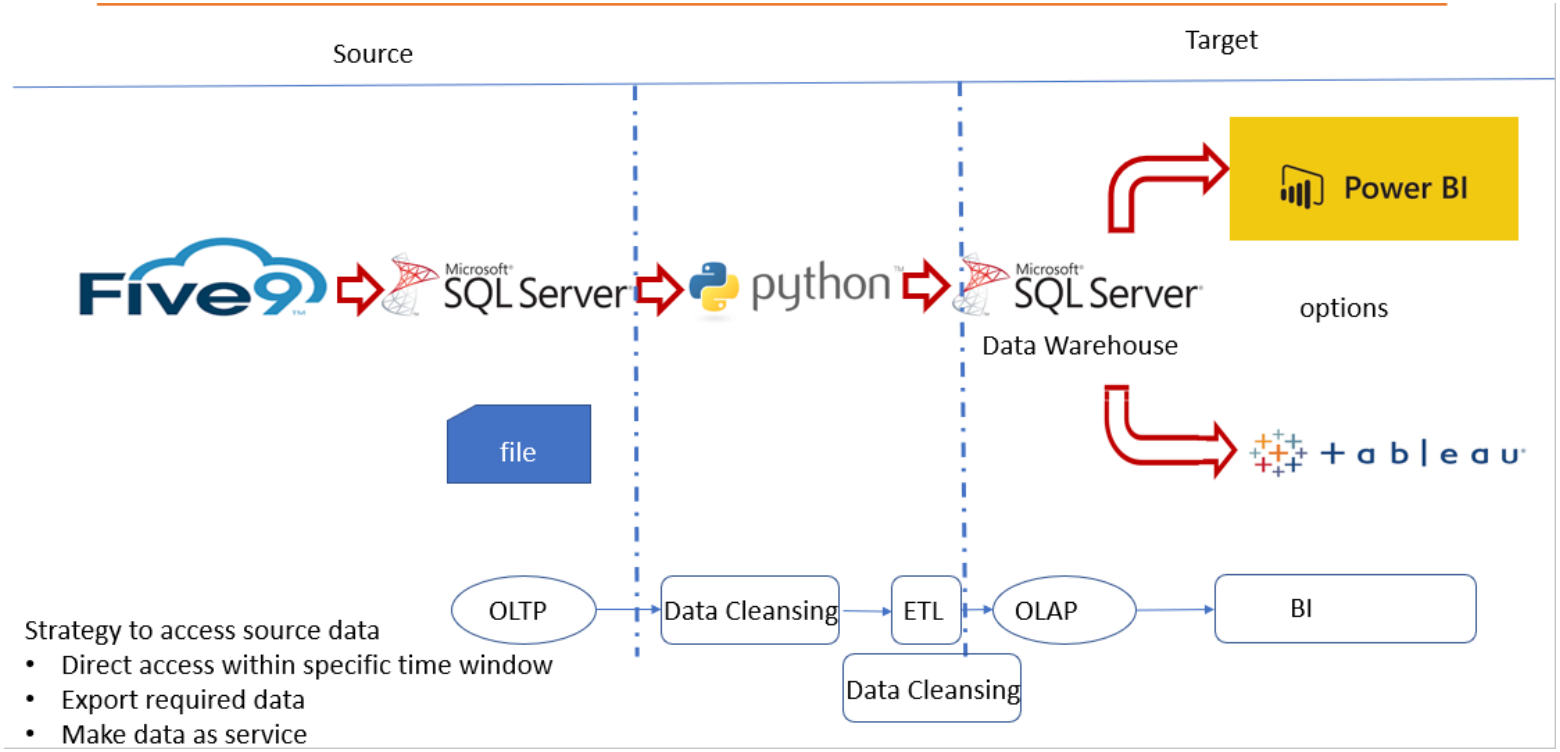
Agenda

- Project Introduction
- Business Requirements
- Data Warehouse Modeling
- Data Cleansing
- Load Data to CCBISDW
- BI Implementation
- Data Validation

Project Introduction
- Project Background

- Call centers have become a major business, provided vast amount of customer information and generated tremendous opportunities.
- CCBIS project aims to analyze a call center data to provide decision makers with valuable insights about customers and their needs in order to
  - Improve the customer experience track (CET)
  - Improve the operation of a call center
  - Improve companies' products and services
  - Predict customer churn

Project Introduction
- Project Architecture



Business Requirements
- Key Performance Indicators

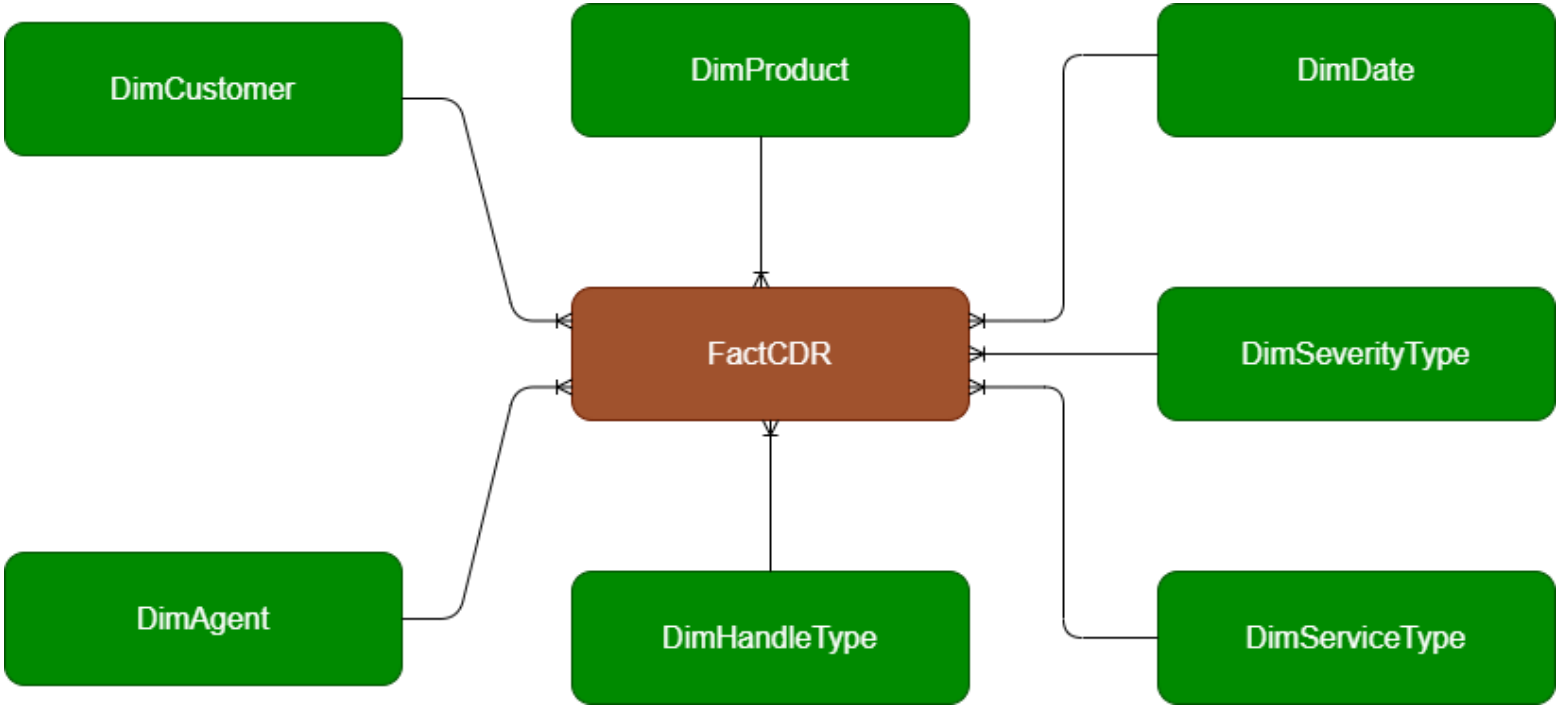
KPI Name	Definition
Open Tickets	Customer’s problem hasn’t been solved
Processed Tickets	Customer's problem has been handled
First Resolved	Customer's problem resolved within the first call
Call Volume	All incoming and outgoing calls handled by the call center over a given period
SLA%	Service Level Agreement
Avg Days to Resolve	Average days to handle a call or transaction
Days Aging for Open Tickets	The number of days that an unsolved ticket has stayed open
NPS Score	On a 0-10 scale: how likely would you recommend this brand to your friends or relatives?
Satisfaction %	Indicator of customer’s happiness

Business Requirements
- Dashboards Design

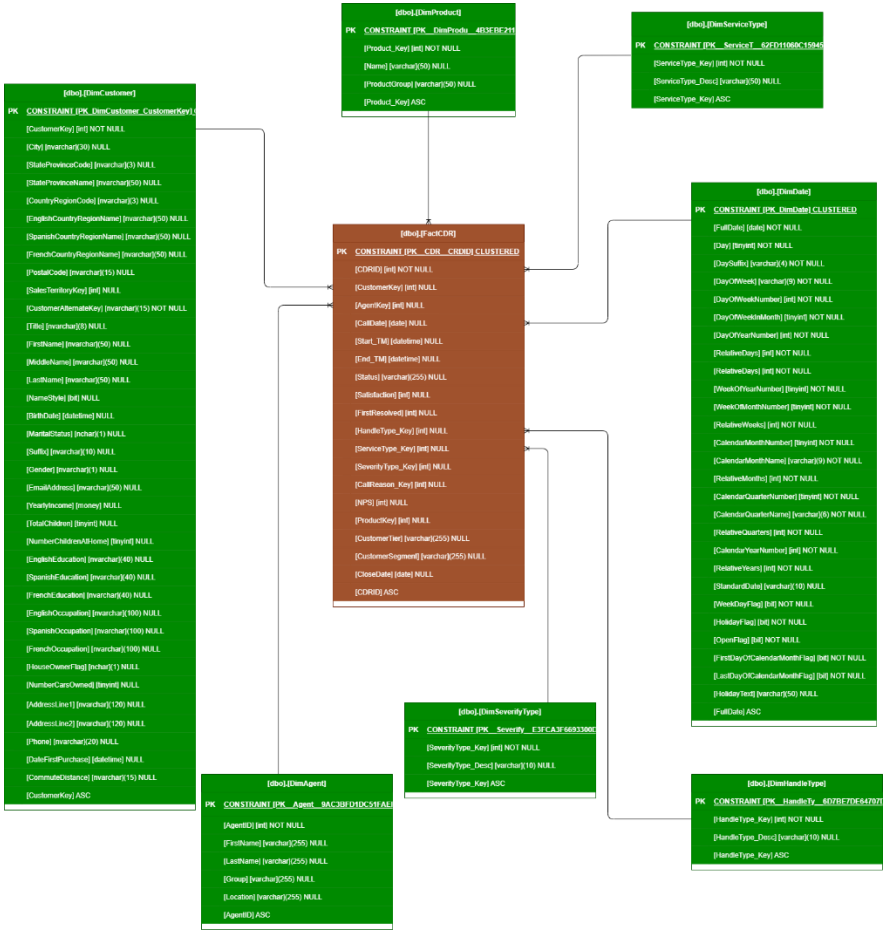
- Executive Dashboard of Call Center Performance
- Customer Service Dashboard
- Agent Effectiveness Dashboard
- Call Volumes Dashboard
- Report with Relative Date Filter
- Report with Last N Weeks based on Report Week
- Customer Churn Dashboards

Data Warehouse Modeling - Conceptual Model

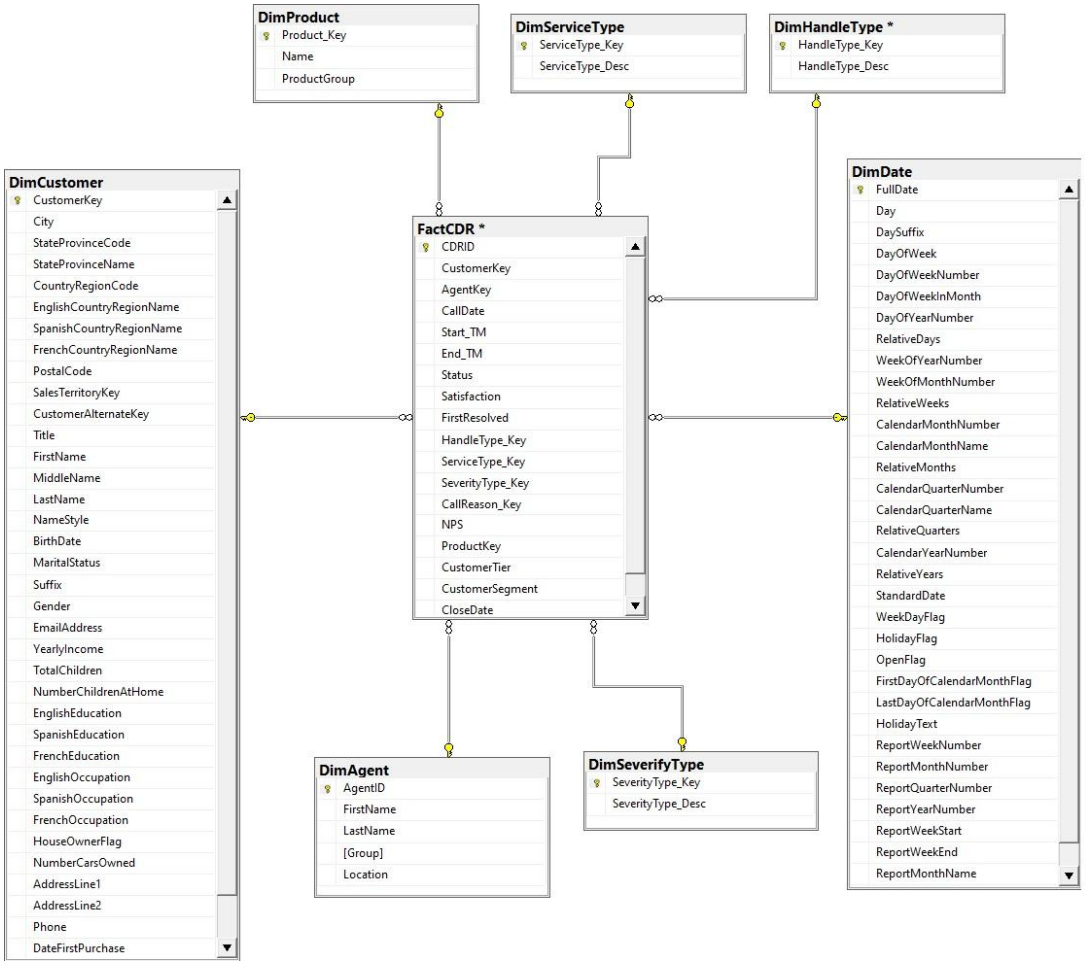
- Star Schema



Data Warehouse Modeling - Logical Model



Data Warehouse Modeling - Physical Model



Data Cleansing

- Handling Missing Titles in DimCustomer

```
null_columns=DimCustomer.columns[DimCustomer.isnull().any()]
DimCustomer[null_columns].isnull().sum()
```

```
Title            18383
MiddleName        7830
Suffix           18481
AddressLine2     18172
```

```
DimCustomer['Title']=DimCustomer['Title'].mask((DimCustomer['Gender'].map(lambda x : x=='F')) &
```

```
DimCustomer['Title']=DimCustomer['Title'].mask((DimCustomer['Gender'].map(lambda x : x=='F')) &
```

```
DimCustomer['Title']=DimCustomer['Title'].mask((DimCustomer['Gender'].map(lambda x : x=='M')),
```

```
DimCustomer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18484 entries, 0 to 18483
Data columns (total 29 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   CustomerKey           18484 non-null  int64
 1   GeographyKey          18484 non-null  int64
 2   CustomerAlternateKey  18484 non-null  object
 3   Title                 18484 non-null  object
```

Data Cleansing Cont.

- Detecting outliers in DimCustomer

Detecting outliers inYearlyIncome column of DimCustomer

```python
import numpy as np
import matplotlib.pyplot as plt
```

```python
q1=np.quantile(DimCustomer.YearlyIncome,0.25)
q3=np.quantile(DimCustomer.YearlyIncome,0.75)
med=np.median(DimCustomer.YearlyIncome)
IQR=q3-q1
upper_bound=q3+(1.5*IQR)
lower_bound=q1-(1.5*IQR)
print(IQR,upper_bound,lower_bound)
```

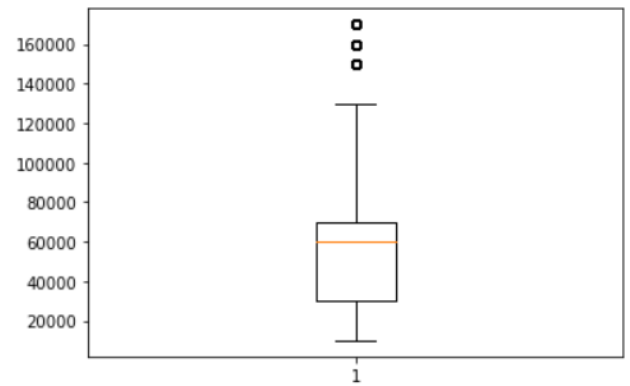
```
40000.0 130000.0 -30000.0
```

```python
outliers=DimCustomer.YearlyIncome[(DimCustomer.YearlyInc
print('The following are outliers in the boxplot: {}'.fo
```

```
The following are outliers in the boxplot: 79       160000.0
80       170000.0
180      160000.0
181      170000.0
240      160000.0
           ...
17960    160000.0
18221    150000.0
18222    160000.0
18238    150000.0
18239    150000.0
Name: YearlyIncome, Length: 309, dtype: float64
```

```python
YearlyIncome=DimCustomer.YearlyIncome
```

```python
plt.boxplot(YearlyIncome)
fig=plt.figure(figsize=(10,7))
plt.show()
```




```
<Figure size 720x504 with 0 Axes>
```

Data Cleansing Cont.
- Detecting outliers in CDR

Detecting outliers in NPS column of CDR

In [67]:

```
q1=np.quantile(CDR.NPS,0.25)
q3=np.quantile(CDR.NPS,0.75)
med=np.median(CDR.NPS)
IQR=q3-q1
upper_bound=q3+(1.5*IQR)
lower_bound=q1-(1.5*IQR)
print(IQR,upper_bound,lower_bound)
```

```
7.0 19.5 -8.5
```

In [68]:

```
outliers=CDR.NPS[(CDR.NPS<lower_bound)|(CDR.NPS>upper_bound)]
print('The following are outliers in the boxplot: {}'.format(outliers))
```

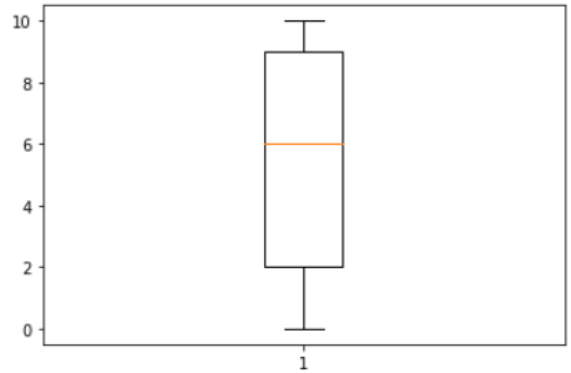
```
The following are outliers in the boxplot: Series([], Name: NPS, dtype: int64)
```

In [69]:

```
NPS=CDR.NPS
```

In [70]:

```
plt.boxplot(NPS)
fig=plt.figure(figsize=(10,7))
plt.show()
```




```
<Figure size 720x504 with 0 Axes>
```

Data Cleansing Cont.

- Correcting the data types in CDR

```python
CDR.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16000 entries, 0 to 15999
Data columns (total 18 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   CDRID             16000 non-null  int64
 1   CustomerKey       16000 non-null  int64
 2   AgentKey          16000 non-null  int64
 3   CallDate          16000 non-null  object
 4   Start_TM          16000 non-null  object
 5   End_TM            16000 non-null  object
 6   Status            16000 non-null  object
 7   Satisfaction      16000 non-null  int64
 8   FirstResolved     16000 non-null  int64
 9   HandleType_Key    16000 non-null  int64
 10  ServiceType_Key   16000 non-null  int64
 11  SeverityType_Key  16000 non-null  int64
 12  CallReason_Key    16000 non-null  int64
 13  NPS               16000 non-null  int64
 14  ProductKey        16000 non-null  int64
 15  CustomerTier      16000 non-null  object
 16  CustomerSegment   16000 non-null  object
 17  CloseDate         16000 non-null  object
```

```python
CDR['CallDate']=pd.to_datetime(CDR['CallDate'])
CDR['Start_TM']=pd.to_datetime(CDR['Start_TM'])
CDR['End_TM']=pd.to_datetime(CDR['End_TM'])
CDR['CloseDate']=pd.to_datetime(CDR['CloseDate'])
```

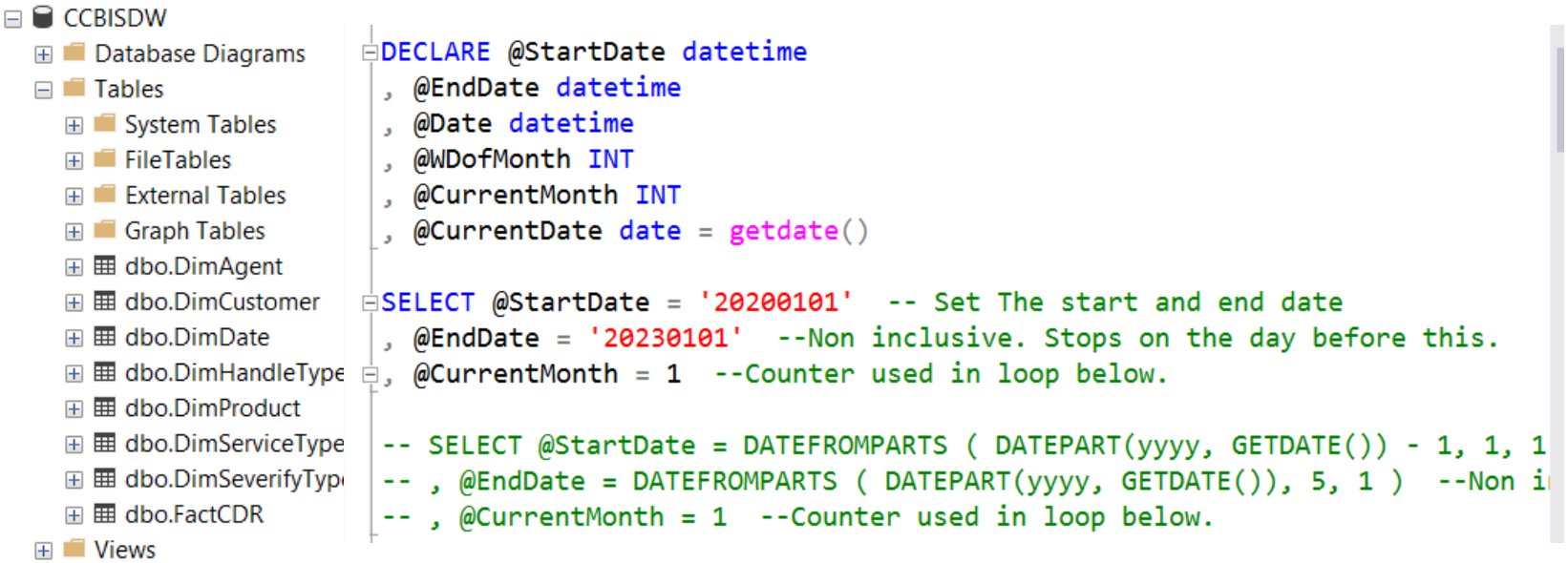
```python
CDR.info()
```

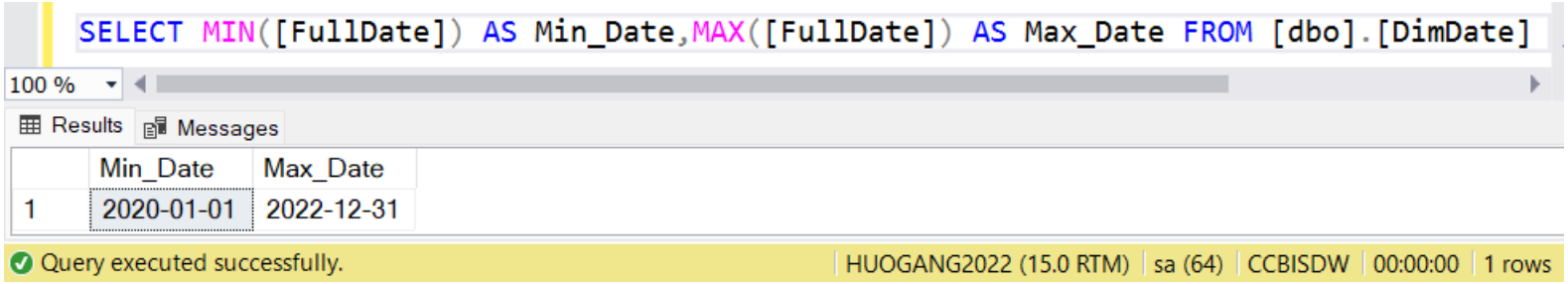
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16000 entries, 0 to 15999
Data columns (total 18 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   CDRID             16000 non-null  int64
 1   CustomerKey       16000 non-null  int64
 2   AgentKey          16000 non-null  int64
 3   CallDate          16000 non-null  datetime64[ns]
 4   Start_TM          16000 non-null  datetime64[ns]
 5   End_TM            16000 non-null  datetime64[ns]
 6   Status            16000 non-null  object
 7   Satisfaction      16000 non-null  int64
 8   FirstResolved     16000 non-null  int64
 9   HandleType_Key    16000 non-null  int64
 10  ServiceType_Key   16000 non-null  int64
 11  SeverityType_Key  16000 non-null  int64
 12  CallReason_Key    16000 non-null  int64
 13  NPS               16000 non-null  int64
 14  ProductKey        16000 non-null  int64
 15  CustomerTier      16000 non-null  object
 16  CustomerSegment   16000 non-null  object
 17  CloseDate         16000 non-null  datetime64[ns]
```

Load Data to CCBISDW

- Customize the Date Range for DimDate

- Select a Date Range for DimDate





Load Data to CCBISDW
- Create a Database Connection

Generate dataframe from CSV

[78]:
```
## From SQL to DataFrame Pandas
import pandas as pd
import pyodbc
import os
import pandas as pd
# !pip install sqlalchemy
import sqlalchemy

from sqlalchemy import create_engine, MetaData, Table, select
from six.moves import urllib

os.chdir(r"../../data/processed")
ccbis_files=os.listdir()
createVar = locals()
for i in ccbis_files:
    if i.endswith("csv"):
        createVar[i.split('.')[0]] = pd.read_csv(i)
```

Create a database connection engine

[79]:
```
params = urllib.parse.quote_plus("DRIVER={ODBC Driver 17 for SQL Server};SERVER=localhost;DATABASE=CCBISDW;UID=sa;PWD=SQLServer
engine = sqlalchemy.create_engine("mssql+pyodbc:///?odbc_connect=%s" % params)
engine.connect()
```

[79]:
```
<sqlalchemy.engine.base.Connection at 0x24441a0cad0>
```

Load Data to CCBISDW Cont.
- Load All Dimension and Fact Tables

Load DimProduct Table

In [7]:
```
DimProductGroup.columns = ['ProductGroup_Key','ProductGroup']
DimPro = pd.merge(DimProduct,DimProductGroup, on='ProductGroup_Key')
DimPro = DimPro[['Product_Key','Name','ProductGroup']]
DimPro.to_sql(name='DimProduct',con=engine, index=False, if_exists='append')
```

Out[7]: -1

Load Dimcustomer Table

In [3]:
```
#DimProductGroup.columns = ['ProductGroup_Key','ProductGroup']
DimCust = pd.merge(DimCustomer,DimGeography, on='GeographyKey')
```

In [4]:
```
DimCust = DimCust.loc[:, DimCust.columns != 'GeographyKey']
```

In [5]:
```
DimCust.to_sql(name='DimCustomer',con=engine, index=False, if_exists='append')
```

Out[5]: -1

Load FactCDR

In [6]:
```
CDR.to_sql(name='FactCDR',con=engine, index=False, if_exists='append')
```

Out[6]: -1

Load Data to CCBISDW Cont.

- Updating the Max(CallDate) of FactCDR

```sql
-- Execute this SQL after load FactCDR
use CCBISDW
declare @MaxCallDay date
declare @Days int
select @MaxCallDay = MAX(CallDate) from [dbo].[FactCDR]
SELECT @Days = DATEDIFF(day, @MaxCallDay,'2022-12-31')

update [dbo].[FactCDR]
set [CallDate] = DATEADD(day, @Days, [CallDate]),
    [Start_TM] = DATEADD(day, @Days, [Start_TM]),
    [End_TM] = DATEADD(day, @Days, [End_TM]),
    [CloseDate] = DATEADD(day, @Days, [CloseDate]);
```

Load Data to CCBISDW Cont.
- Max Date of DimDate vs. FactCDR

```sql
USE CCBISDW
GO

SELECT MIN([FullDate]) AS Min_DimDate, MAX([FullDate]) AS Max_DimDate
FROM [dbo].[DimDate];

SELECT MIN([CallDate]) AS Min_CallDate, MAX([CallDate]) AS Max_CallDate
FROM [dbo].[FactCDR]
```

100 %

Results | Messages

	Min_DimDate	Max_DimDate
1	2020-01-01	2022-12-31

	Min_CallDate	Max_CallDate
1	2022-05-19	2022-12-31

Query executed successfully. | HUOGANG2022 (15.0 RTM) | sa (64) | CCBISDW | 00:00:00 | 2 rows

Load Data to CCBISDW Cont.

- Data Validation for DimCustomer

```sql
USE CCBIS
GO
SELECT 'CCBIS DimCustomer Total Rows Counts', COUNT(*)
FROM [dbo].[DimCustomer]
UNION
SELECT 'Distinct CustomerKey Counts',COUNT(DISTINCT [CustomerKey])
FROM [dbo].[DimCustomer];

USE CCBISDW
GO
SELECT 'CCBISDW DimCustomer Total Rows Counts', COUNT(*)
FROM [dbo].[DimCustomer]
UNION
SELECT 'Distinct CustomerKey Counts',COUNT(DISTINCT [CustomerKey])
FROM [dbo].[DimCustomer];
```

100 %

Results | Messages

	(No column name)	(No column name)
1	CCBIS DimCustomer Total Rows Counts	18484
2	Distinct CustomerKey Counts	18484

	(No column name)	(No column name)
1	CCBISDW DimCustomer Total Rows Counts	18484
2	Distinct CustomerKey Counts	18484

Query executed successfully. | HUOGANG2022 (15.0 RTM) | sa (67) | CCBISDW | 00:00:00 | 4 rows

Load Data to CCBISDW Cont.
- Data Validation for FactCDR

```sql
USE CCBIS
GO
SELECT 'CCBIS CDR Total Rows Counts', COUNT(*) FROM  [dbo].[CDR]
UNION
SELECT 'Distinct CDRID Counts', COUNT(DISTINCT [CDRID]) FROM [dbo].[CDR]
UNION
SELECT 'Distinct CustomerKey Counts', COUNT(DISTINCT [CustomerKey]) FROM [dbo].[CDR]
UNION
SELECT 'Distinct AgentKey Counts',COUNT(DISTINCT [AgentKey]) FROM [dbo].[CDR];

USE CCBISDW
GO
SELECT 'CCBISDW FactCDR Total Rows Counts', COUNT(*) FROM [dbo].[FactCDR]
UNION
SELECT 'Distinct CDRID Counts', COUNT(DISTINCT [CDRID]) FROM [dbo].[FactCDR]
UNION
SELECT 'Distinct CustomerKey Counts', COUNT(DISTINCT [CustomerKey]) FROM [dbo].[FactCDR]
UNION
SELECT 'Distinct AgentKey Counts',COUNT(DISTINCT [AgentKey]) FROM [dbo].[FactCDR];
```

90 %

Results | Messages

	(No column name)	(No column name)
1	CCBIS CDR Total Rows Counts	16000
2	Distinct CDRID Counts	16000
3	Distinct CustomerKey Counts	3955
4	Distinct AgentKey Counts	100

	(No column name)	(No column name)
1	CCBISDW FactCDR Total Rows Counts	16000
2	Distinct CDRID Counts	16000
3	Distinct CustomerKey Counts	3955
4	Distinct AgentKey Counts	100

Query executed successfully. | HUOGANG2022 (15.0 RTM) | sa (67) | CCBISDW | 00:00:00 | 8 rows

Load Data to CCBISDW Cont.

- Create View dbo.CustomerService

- Create View dbo.CustomerService in CCBISDW and export the data to Excel for creating Tableau dashboard

```
df.to_excel(r"C:\Victoria Courses\DA Projects\HG DA Project Work\02-Telecom\
```

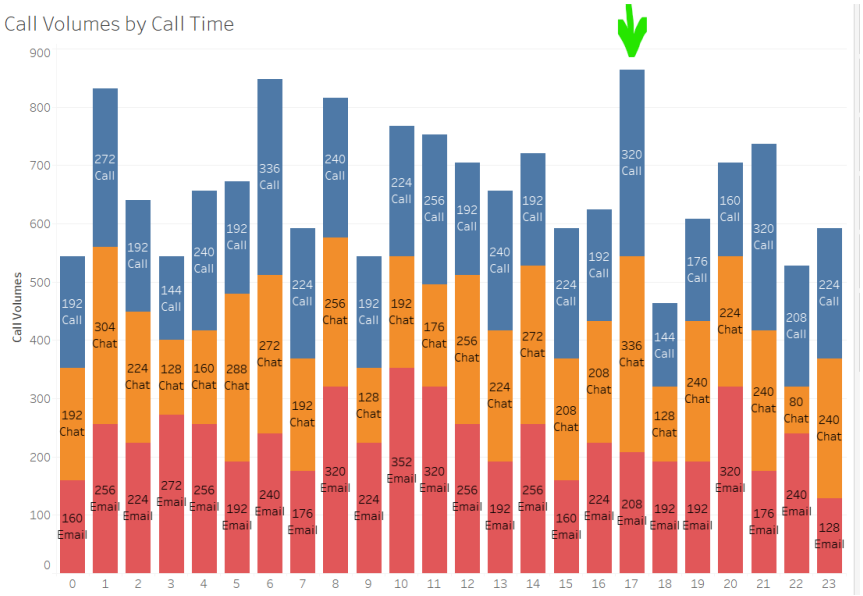
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16000 entries, 0 to 15999
Data columns (total 23 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   CDRID                16000 non-null  int64
 1   CallDate             16000 non-null  object
 2   Start_TM             16000 non-null  datetime64[ns]
 3   End_TM               16000 non-null  datetime64[ns]
 4   CloseDate            16000 non-null  object
 5   Status               16000 non-null  object
 6   Satisfaction         16000 non-null  int64
 7   FirstResolved        16000 non-null  int64
 8   NPS                  16000 non-null  int64
 9   CustomerTier         16000 non-null  object
 10  CustomerSegment      16000 non-null  object
 11  Customer_PostalCode  16000 non-null  object
 12  Customer_Country     16000 non-null  object
 13  SeverityType         16000 non-null  object
 14  ServiceType          16000 non-null  object
 15  HandleType           16000 non-null  object
 16  Agent                16000 non-null  object
 17  Group                16000 non-null  object
 18  Product              16000 non-null  object
 19  ProductGroup         16000 non-null  object
 20  ReportWeek           16000 non-null  object
 21  ReportWeekNumber     16000 non-null  int64
 22  DaystoResolved       13767 non-null  float64
```

BI Implementation
- Using Tableau and Power BI

- ✓ Executive Dashboard of Call Center Performance
- ✓ Customer Service Dashboard
- ✓ Agent Effectiveness Dashboard
- ✓ Call Volumes Dashboard
- ✓ Report with Relative Date Filter
- ✓ Report with Last N Weeks based on Report Week
- ✓ Executive Dashboard with Multi Custom Date Range

Data Validation
- Call Volumes by Call Time




```
SELECT 'Call', COUNT(CDRID) FROM [dbo].[CustomerService]
WHERE DATEPART(hour,[Start_TM])=17 AND [HandleType]='Call'
UNION
SELECT 'Chat', COUNT(CDRID) FROM [dbo].[CustomerService]
WHERE DATEPART(hour,[Start_TM])=17 AND [HandleType]='Chat'
UNION
SELECT 'Email', COUNT(CDRID) FROM [dbo].[CustomerService]
WHERE DATEPART(hour,[Start_TM])=17 AND [HandleType]='Email'
```

90 %

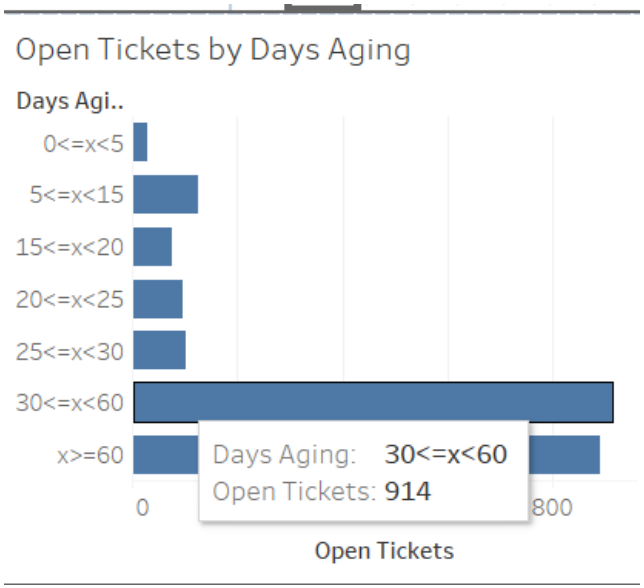
Results | Messages

	(No column name)	(No column name)
1	Call	320
2	Chat	336
3	Email	208

Query executed su... | HUOGANG2022 (15.0 RTM) | sa (68) | CCBISDW | 00:00:00 | 3 rows

Data Validation Cont.

- Open Tickets by Days Aging




```sql
SELECT COUNT([CDRID]) AS [Open Tickets (30=<x<60)]
FROM [dbo].[CustomerService]
WHERE  [Status]='Open' AND
DATEDIFF(day,[CallDate],[CloseDate]) BETWEEN 30 AND 59
```

90 %

Results | Messages

	Open Tickets (30=<x<60)
1	914

Query execut... | HUOGANG2022 (15.0 RTM) | sa (67) | CCBISDW | 00:00:00 | 1 rows

Data Validation

- BI Report by Last 5 Months

Relative Date

Group		
Level 1	% CSAT	86.29%
	NPS Score	-20
	Avg. Days Aging for Open Tickets	52
	Open Tickets	426
Level 2	% CSAT	77.05%
	NPS Score	-11
	Avg. Days Aging for Open Tickets	50
	Open Tickets	204
Level 3	% CSAT	78.88%
	NPS Score	-17
	Avg. Days Aging for Open Tickets	50
	Open Tickets	490
Level 4	% CSAT	83.16%
	NPS Score	-22
	Avg. Days Aging for Open Tickets	48
	Open Tickets	535
Level 5	% CSAT	80.45%
	NPS Score	-22
	Avg. Days Aging for Open Tickets	49
	Open Tickets	578

Call Date

Last 5 months

Years | Quarters | Months | Weeks | Days

Previous month | Last 5 months

Anchor month | Next 3 months

Next month | Month to anchor

2022-08-01 to 2022-12-31

Data Validation Cont.

- BI Report by Last 5 Months

86.29%
-20
52
426
77.05%
-11
50
204
78.88%
-17
50
490
83.16%
-22
48
535
80.45%
-22
49
578

```sql
SELECT 'Leve1_CAST%', CAST(SUM(CASE WHEN [Satisfaction]=5 THEN 1 ELSE 0 END) AS FLOAT)*100/CAST(COUNT([CDRID])AS FLOAT)
FROM [dbo].[CustomerService]
WHERE ([CallDate] BETWEEN '20220801' AND '20221231') AND ([Group]='Level 1')
UNION
SELECT 'Leve2_CAST%', CAST(SUM(CASE WHEN [Satisfaction]=5 THEN 1 ELSE 0 END) AS FLOAT)*100/CAST(COUNT([CDRID])AS FLOAT)
FROM [dbo].[CustomerService]
WHERE ([CallDate] BETWEEN '20220801' AND '20221231') AND ([Group]='Level 2')
UNION
SELECT 'Leve3_CAST%', CAST(SUM(CASE WHEN [Satisfaction]=5 THEN 1 ELSE 0 END) AS FLOAT)*100/CAST(COUNT([CDRID])AS FLOAT)
FROM [dbo].[CustomerService]
WHERE ([CallDate] BETWEEN '20220801' AND '20221231') AND ([Group]='Level 3')
UNION
SELECT 'Leve4_CAST%', CAST(SUM(CASE WHEN [Satisfaction]=5 THEN 1 ELSE 0 END) AS FLOAT)*100/CAST(COUNT([CDRID])AS FLOAT)
FROM [dbo].[CustomerService]
WHERE ([CallDate] BETWEEN '20220801' AND '20221231') AND ([Group]='Level 4')
UNION
SELECT 'Leve5_CAST%', CAST(SUM(CASE WHEN [Satisfaction]=5 THEN 1 ELSE 0 END) AS FLOAT)*100/CAST(COUNT([CDRID])AS FLOAT)
FROM [dbo].[CustomerService]
WHERE ([CallDate] BETWEEN '20220801' AND '20221231') AND ([Group]='Level 5')
```

90 %

Results | Messages

	(No column name)	(No column name)
1	Leve1_CAST%	86.2859931673987
2	Leve2_CAST%	77.0491803278689
3	Leve3_CAST%	78.8789601949634
4	Leve4_CAST%	83.1615120274914
5	Leve5_CAST%	80.4516584333098

Query executed successfully. | HUOGANG2022 (15.0 RTM) | sa (67) | CCBISDW | 00:00:00 | 5 rows

CCBIS Project Part 2 – Customer Churn Prediction

Presented by Gang Huo

March 11, 2026

Agenda

- Project Background
- Data Preprocessing for ML
- ML Data Modeling and Testing
- Data Visualization in Power BI
- Recommendations

Project Background
- Importance of Churn Prediction

- Customer churn refers to when customers or subscribers switch to another company or service provider.
- Customer churn is a major concern for the telecom industry because it's much more expensive to acquire a new customer than to retain an old one in this extremely competitive sector.
- Customer churn prediction can help the subscription based businesses like telecom companies improve customer retention and revenue.

Project Background
- Data Overview

- The Telco-Customer-Churn table contains information on all 7,043 customers from a telecom company in Ontario in Q3 2022.
- Each record represents one customer, and contains details about their demographics, account information, subscription services and churn label (Yes/No) for the quarter.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

Project Background
- Main Tasks

- Create churn analysis dashboards in Power BI to distill actionable insights.
- Build a machine learning model to predict the churn probabilities of new customers.
- Create a prediction dashboard to allow user to review new customers according to churn probability.

Data Preprocessing for ML

- Data Cleansing
- Splitting the dataset
- Encoding the categorical data
- Feature scaling (standardization)

ML Data Modeling and Testing
- Machine Learning Algorithms

- Logistic Regression
- Gradient Boosting
- Decision Tree
- Random Forest

Data Visualization in Power BI

- Churn Overview Dashboard
- Churn Analysis Dashboard
- Churn Prediction Dashboard
- Churn Probability Dashboard

Recommendations

- Contract is key. Identify the customers with monthly contracts and provide them with incentives to move to a two year contract or one year contract.
- Examine issues associated with the fiber optic network to see if performance issues bring about poor customer experience.
- Improve the services in terms of technical support and online backup to maximize the customer satisfaction and loyalty.